

Legion Swarm & Sovereign Engine Architecture

Here is the complete architectural mapping of your JSONL logs, DuckDB/LanceDB databases, Ray Swarm Actors, and the C++/Numba DSP processing engines.

```
```mermaid
graph TD
 %% Styling
 classDef cold fill:#2c3e50,stroke:#7f8c8d,stroke-
width:2px,color:#fff;
 classDef hot fill:#2980b9,stroke:#3498db,stroke-width:2px,color:#fff;
 classDef engine fill:#8e44ad,stroke:#9b59b6,stroke-
width:2px,color:#fff;
 classDef client fill:#27ae60,stroke:#2ecc71,stroke-
width:2px,color:#fff;

 %% Nodes
 subgraph Disk ["Cold Storage & Raw Files"]
 AudioFiles["Raw Catalog Stems (.wav/.mp3)
• 393,419 Audio
Files on E-Drive"]:::cold
 JSONL["JSONL Shards (E-Drive)
• 393,419 File Maps (~155
MB)"]:::cold
 DuckDB["DuckDB (sonic_core_v2.duckdb)
• 1,084 Audio
Features
• 781,659 System Paths"]:::cold
 LanceDB["LanceDB Space
• 302 Code Vectors
• 1,083 Audio
Vectors"]:::cold
 end

 subgraph Processing ["DSP & Feature Extraction"]
 Pedalboard["Pedalboard C++ Engine
• Extracts RMS & Crest
Factor
• Runs outside Ray (Fork-Safe)"]:::engine
 end

 subgraph RayCluster ["Ray Cluster Shared Memory (Zero-Copy Plasma
Store)"]
 ArrowSwarm["SwarmKnowledgeRegistry Actor
• 27 Registered
Arrow Tables
• Holds Audio/Metadata in RAM"]:::hot
 CodeSwarm["CodeSwarmKnowledgeRegistry Actor
• 30 Registered
Arrow Tables
• Holds Code/GDU maps in RAM"]:::hot
 DSPAlign["DSPAlignmentActor (Stateful)
• One actor per CPU
core (Numba/SciPy)
• Matches track segments to baselines"]:::hot
 SovActor["SovereignEngine Actor
• Trains Random Forest &
Isolation Forest
• Boots in 1.2s -> Queries in <10ms"]:::engine
 end

 subgraph Consumers ["Swarm Clients (MCP & Agents)"]
 MCP["MCP RAG Server
(mcp_rag_server.py)"]:::client
 Agent["Langgraph ONNX
Agent
(langgraph_onnx_agent.py)"]:::client
 Ableton["Ableton Integration Stems"]:::client
 end

 %% Connections
```

```

AudioFiles -->|Raw Decoding| Pedalboard
Pedalboard -->|Pydantic Firewall| DSPAlign
JSONL -->|Parallel Ingest| ArrowSwarm
DuckDB -->|Metadata Tables| ArrowSwarm
DuckDB -->|Feature Vectors| SovActor
LanceDB -->|Code Maps| CodeSwarm
LanceDB -->|Audio Vectors| DSPAlign

ArrowSwarm -->|In-Memory Training Data| SovActor
ArrowSwarm -->|Zero-Copy RecordBatches| DSPAlign

DSPAlign -->|Verified Alignments| Agent
SovActor -->|10ms Inference / Diagnostics| MCP
SovActor -->|KNN Alignments| Agent
ArrowSwarm -->|RAG Queries| MCP
CodeSwarm -->|Semantic Code Search| MCP
MCP --> Ableton
...

Data Size & Memory Mapping Breakdown

1. File Catalog & Raw Audio
* **Total Catalog Size**: **393,419 tracks** mapped from the E-Drive.
* **Physical Footprint**: **~155 MB** across 8 `.jsonl` files.
* **Raw Audio**: Decoded via **Pedalboard C++** to extract RMS and Crest Factor before passing values to Ray, avoiding process forking crashes.

2. Databases (DuckDB & LanceDB)
* **DuckDB (`web_intel_sonicdb.duckdb`)**:
 * `semantic_map`: **393,419 rows** (100% metadata coverage of the JSONL shards).
 * `computer_fs`: **770,411 rows** (entire filesystem registry).
 * `audio_features`: **1,084 rows** (the actual processed DSP profiles).
* **LanceDB (`lancedb_web_intel_rag`)**:
 * `mined_code_vectors`: **302 vectors** (vectorized Python symbols).
 * `chris_lake_speed_test`: **1,083 vectors** (audio vector space used by DSPAlignmentActor).

3. Ray Shared Memory Space
* **Arrow Swarm Registry**: Holds 27 memory-mapped PyArrow Tables in the Plasma Object Store for zero-copy queries.
* **Code Swarm Registry**: Holds 30 code structure/GDU tables in memory.
* **DSPAlignmentActor**:
 * Fitted with reference baselines (e.g. "Chris Lake" features).
 * Performs SciPy/Numba Euclidean distance calculations on incoming segments dynamically.
* **SovereignEngine Actor**:

```

- \* Pulls the \*\*1,084 DSP profiles\*\* (41 features each) from the registry/database.
- \* Trains and holds `RandomForestClassifier` and `IsolationForest` models.
- \* Maintains the `NearestNeighbors` tree in RAM for instant KNN matching (Depths 1-41).